Ch 5 – Part 5

Real time scheduling



Real-Time System

- "A real-time system is a software system that maintains an ongoing and timely interaction with its environment”
  - » Bran Selic

- Real-Time: Pertaining to the performance of a computation during the actual time that the related physical process transpires in order that the results of the computation can be used in guiding the physical process. [IEEE Standard Dictionary]





Types of Real-Time Systems

- Hard real time
  - Timing requirement must always be met (e.g. some reactor trips)
  - Missing deadlines means system failure
- Soft real time
  - Timeliness of events is important but not necessarily critical.
  - Occasionally missing a deadline does not result in system failure (e.g. RT-video, user interface, data logging, even many control loops!)





Types of Real-Time Systems

- Real-time systems are frequently classified as hard or soft real-time systems.
  - A hard real-time system has time-critical deadlines that must be met; otherwise a catastrophic system failure can occur.
  - In a soft real-time system, it is considered undesirable, but not catastrophic, if deadlines are occasionally missed.
- Safety Critical
  - Missing deadlines can cause serious loss





Overview of Real-Time Systems

- A real-time system requires that results be produced within a specified deadline period
- An embedded system is a computing device that is part of a larger system (I.e. automobile, airliner)
- A safety-critical system is a real-time system with catastrophic results in case of failure
- A hard real-time system guarantees that real-time tasks be completed within their required deadlines
- A soft real-time system provides priority of real-time tasks over non real-time tasks





Characteristics of Real-Time Systems

- Real-time systems tend to be:
  - Embedded Systems: i.e. a component of a larger hardware/software system.
  - Interact with an external “environment” (i.e. the “plant” or system under control)
  - Reactive Systems: event-driven and must respond to external stimuli (“events”). System response is typically state dependent.
  - Concurrent Processing: many events that need to be processed in parallel.
  - Safety Critical





Real-Time CPU Scheduling

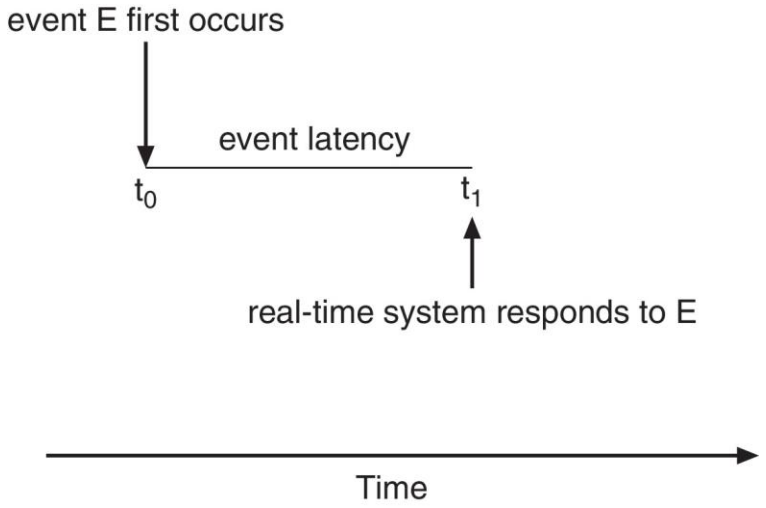
- Can present obvious challenges
- **Soft real-time systems** – Critical real-time tasks have the highest priority, but no guarantee as to when tasks will be scheduled
- **Hard real-time systems – task must be serviced by its deadline**





Real-Time CPU Scheduling

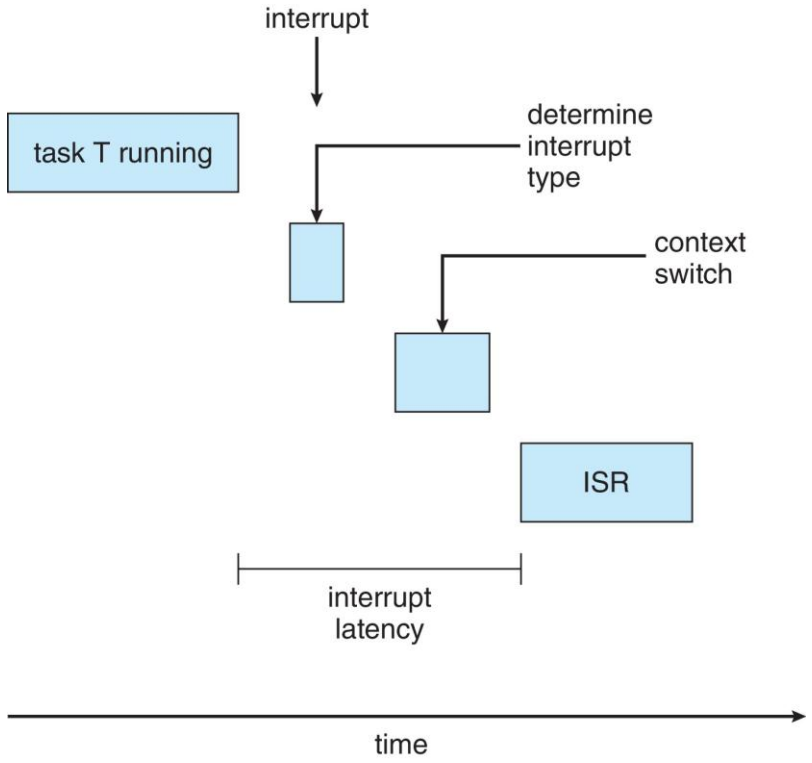
- Event latency – the amount of time that elapses from when an event occurs to when it is serviced.
- Two types of latencies affect performance
  1. **Interrupt latency** – time from arrival of interrupt to start of routine that services interrupt
  2. **Dispatch latency** – time for schedule to take current process off CPU and switch to another







Interrupt Latency

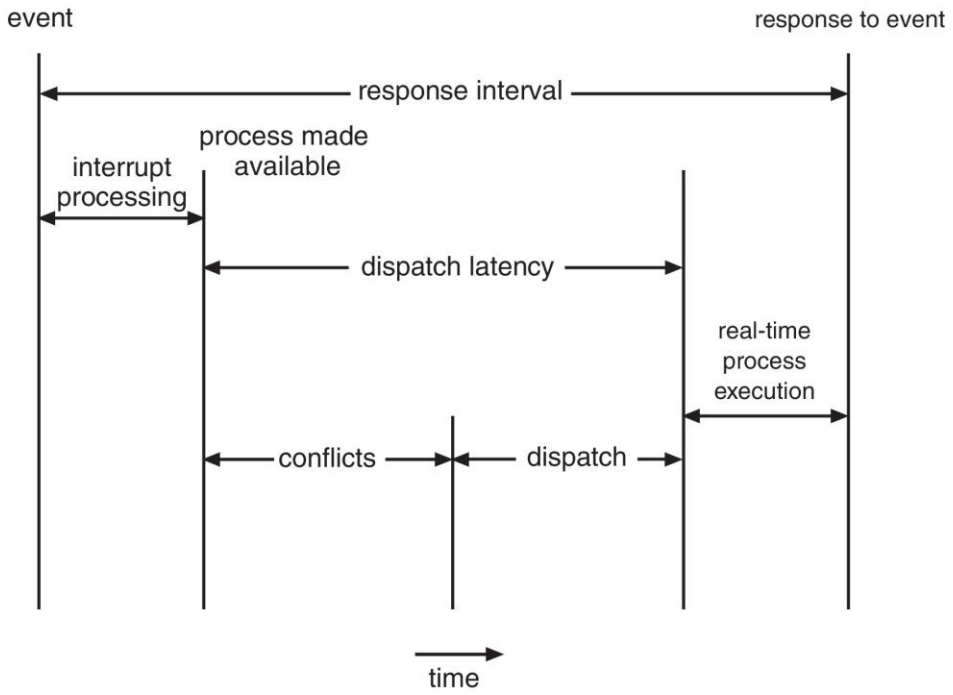






Dispatch Latency

- Conflict phase of dispatch latency:
  1. Preemption of any process running in kernel mode
  2. Release by low-priority process of resources needed by high-priority processes

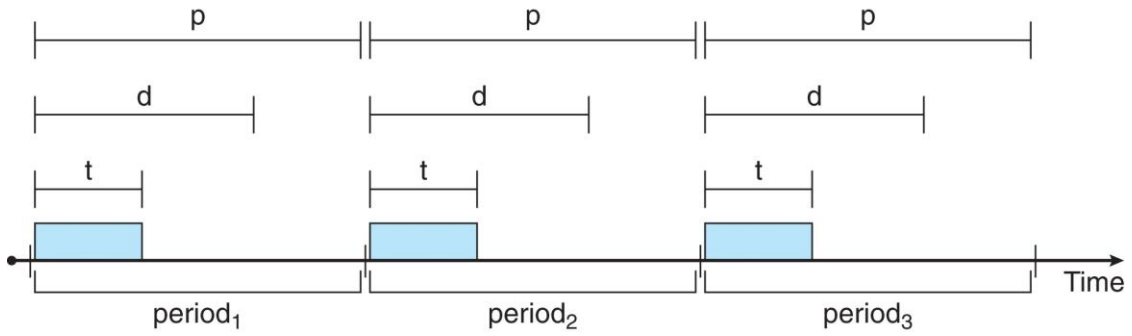






Priority-based Scheduling

- For real-time scheduling, scheduler must support preemptive, priority-based scheduling
  - But only guarantees soft real-time
- For hard real-time must also provide ability to meet deadlines
- Processes have new characteristics: **periodic** ones require CPU at constant intervals
  - Has processing time *t*, deadline *d*, period *p*
  - $0 \leq t \leq d \leq p$
  - **Rate** of periodic task is $1/p$

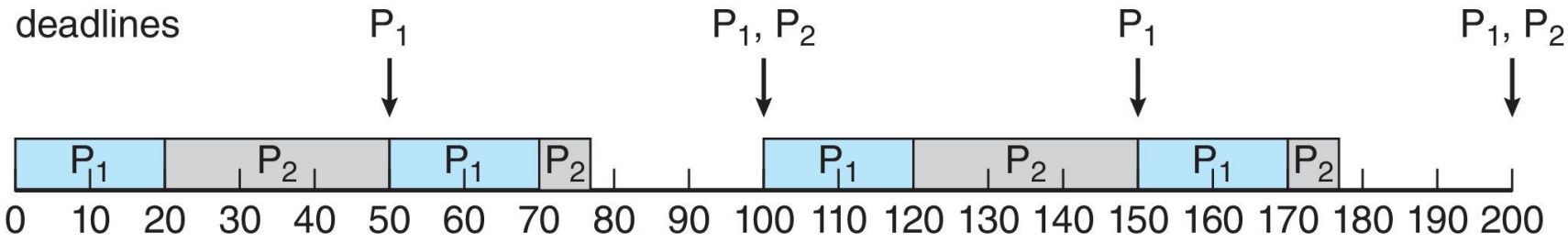






Rate Monotonic Scheduling

- A priority is assigned based on the inverse of its period
- Shorter periods = higher priority;
- Longer periods = lower priority
- P_1 is assigned a higher priority than P_2.

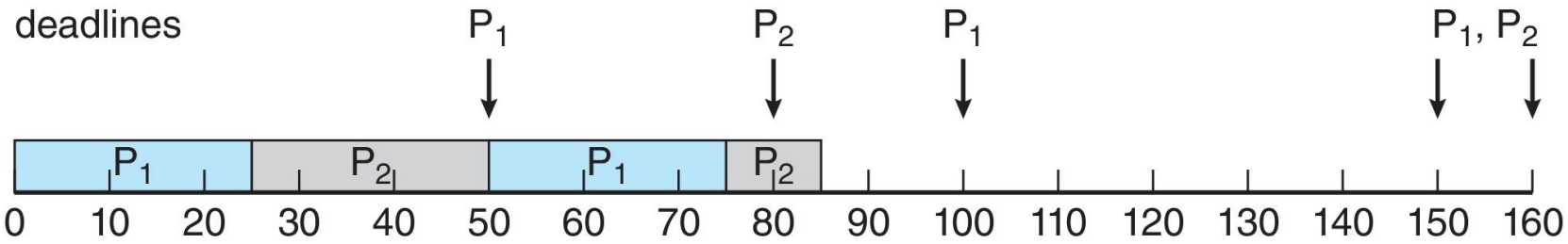






Missed Deadlines with Rate Monotonic Scheduling

Process P2 misses finishing its deadline at time 80





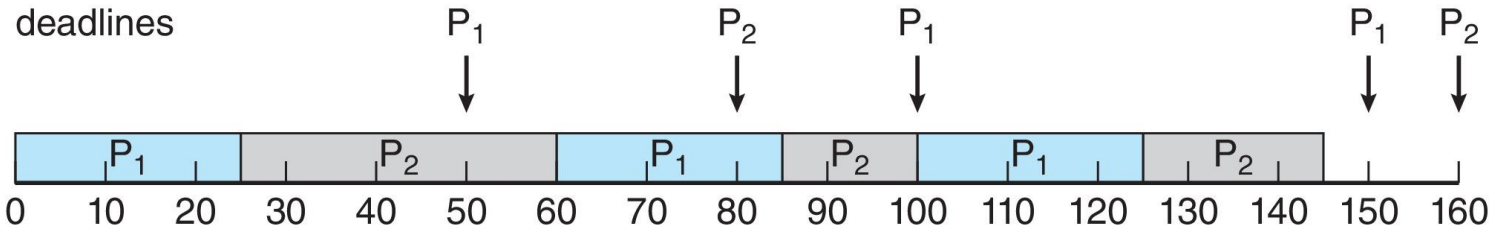


Earliest Deadline First Scheduling (EDF)

- Priorities are assigned according to deadlines:

  the earlier the deadline, the higher the priority;

  the later the deadline, the lower the priority







Proportional Share Scheduling

- *T* shares are allocated among all processes in the system
- An application receives *N* shares where $N < T$
- This ensures each application will receive $\boldsymbol{N} / T$ of the total processor time





POSIX Real-Time Scheduling

- The POSIX.1b standard
- API provides functions for managing real-time threads
- Defines two scheduling classes for real-time threads:
  1. SCHED_FIFO - threads are scheduled using a FCFS strategy with a FIFO queue. There is no time-slicing for threads of equal priority
  2. SCHED_RR - similar to SCHED_FIFO except time-slicing occurs for threads of equal priority
- Defines two functions for getting and setting scheduling policy:
  1. `pthread_attr_getsched_policy(pthread_attr_t *attr, int *policy)`
  2. `pthread_attr_setsched_policy(pthread_attr_t *attr, int policy)`





POSIX Real-Time Scheduling API

```
#include <pthread.h>
#include <stdio.h>
#define NUM_THREADS 5
int main(int argc, char *argv[])
{
   int i, policy;
   pthread_t tid[NUM_THREADS];
   pthread_attr_t attr;
   /* get the default attributes */
   pthread_attr_init(&attr);
   /* get the current scheduling policy */
   if (pthread_attr_getschedpolicy(&attr, &policy) != 0)
      fprintf(stderr, "Unable to get policy.\n");
   else {
      if (policy == SCHED_OTHER) printf("SCHED_OTHER\n");
      else if (policy == SCHED_RR) printf("SCHED_RR\n");
      else if (policy == SCHED_FIFO) printf("SCHED_FIFO\n");
   }
```





POSIX Real-Time Scheduling API (Cont.)

```
   /* set the scheduling policy - FIFO, RR, or OTHER */
   if (pthread_attr_setschedpolicy(&attr, SCHED_FIFO) != 0)
      fprintf(stderr, "Unable to set policy.\n");
   /* create the threads */
   for (i = 0; i < NUM_THREADS; i++)
      pthread_create(&tid[i],&attr,runner,NULL);
   /* now join on each thread */
   for (i = 0; i < NUM_THREADS; i++)
      pthread_join(tid[i], NULL);
}

/* Each thread will begin control in this function */
void *runner(void *param)
{
   /* do some work ... */
   pthread_exit(0);
}
```

